# Miniprojeto ADS

## Medição ativa com iperf com apoio de medição passiva(tcpdump)

26 de março de 2026

Igor Budag, Ricardo Wagner

# Sumário



1 Objetivo ..... 3
2 Variáveis ..... 6
3 Metodologia ..... 10
4 Resultados ..... 24
5 Conclusão ..... 28

# 1 Objetivo

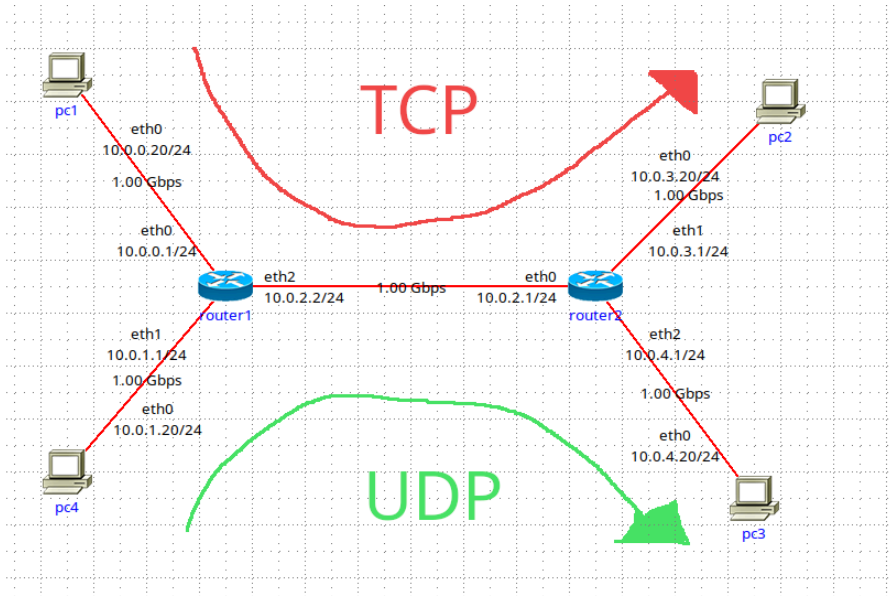
# Objetivo



A proposta desse miniprojeto consiste em analisar uma situação de transmissão tcp com uma carga UDP intensa passando simultaneamente pelo mesmo link (congestionamento). Para análise, foi feito um experimento que terá parâmetros fixados, variáveis de saída e também fatores que serão variados. O experimento deve ser realizado com 8 repetições para cada configuração e deve se fazer todas as combinações possíveis de fatores. No nosso caso $2^2$, já que temos 2 fatores.

# Objetivo




TCP
UDP
pc1
eth0
10.0.0.20/24
1.00 Gbps
eth0
10.0.0.1/24
eth2
10.0.2.2/24
1.00 Gbps
eth0
10.0.2.1/24
router1
router2
eth1
10.0.1.1/24
1.00 Gbps
eth0
10.0.1.20/24
pc4
pc2
eth0
10.0.3.20/24
1.00 Gbps
eth1
10.0.3.1/24
eth2
10.0.4.1/24
1.00 Gbps
eth0
10.0.4.20/24
pc3

# 2 Variáveis

# Variáveis de Saída



Vazão Média do Fluxo TCP (iperf)

Eficiência do Fluxo TCP (tshark)

# Variáveis de Entrada (fatores)



| | |
|---|---|
| Algoritimo TCP | Reno/Cubic |
| BER no enlace entre roteadores | $10^{-6}/10^{-5}$ |

# Parâmetros Fixados



| | |
|---|---|
| Enlace Ethernet | 1 [Gbps] |
| Janela TCP default | 128 [KByte] |
| Duração do teste | 30 [s] |
| Carga UDP | 900 [Mbps] |
| Número de repetições | 32 |

# 3 Metodologia

# Script utilizado



Neste trabalho, foi utilizado o software IMUNES para a criação da simulação da rede. Além disso, empregamos a linguagem Python para automatizar os processos de coleta e análise de dados. Essas duas funções foram organizadas em códigos distintos: o script.py, responsável pela coleta, e o analise.py, dedicado à análise dos dados.

```
# ==================================================================
# Parâmetros do experimento (AUTOMAÇÃO TOTAL)
# ==================================================================
algoritmos = ["reno", "cubic"]

bers_para_testar = [1e-5, 1e-6]

udp_load = 900  # Mbps
repeticoes = 8  # Número de repetições por combinação
# ==================================================================
```

# script.py



```
# ============================
# Execução Principal
# ============================
nome_csv = "resultados_finais_completos.csv"
nome_txt = "iperf_logs_completos.txt"

with open(nome_csv, "w", newline='') as csvfile, \
     open(nome_txt, "w") as f_iperf:

    writer = csv.writer(csvfile)
    writer.writerow(["Algoritmo", "BER", "Repeticao", "Bytes_TCP", "Bytes_Totais",
"Eficiencia", "Vazao_TCP_Mbps"])

    for ber_atual in bers_para_testar:
        # Formata para texto bonito
        ber_texto = f"10^{int(math.log10(ber_atual))}" if ber_atual > 0 else "0"
```

# script.py (ii)



```
    print(f"\n{'#'*60}")
    print(f"🚀 INICIANDO BATERIA PARA BER = {ber_texto}")
    print(f"{'#'*60}")

    # Injeta a perda de pacotes direto no Linux do PC4!
    set_packet_loss(ber_atual)

    for alg in algoritmos:
        print(f"\n{'='*40}")
        print(f"=== Teste: {alg} | BER: {ber_texto} ===")
        print(f"{'='*40}")

        set_tcp(alg)

        for repeticao in range(1, repeticoes + 1):
```

# script.py (iii)



```
            print(f"\n--- Repetição {repeticao} ---")

            reset_tcp_metrics()

            # Mantivemos o número decimal normal pro nome do arquivo não bugar no
Linux
            pcap_file = f"fluxo_{alg}_{ber_atual}_{repeticao}.pcap"

            print("Iniciando tcpdump...")
            start_tcpdump(pcap_file)
            time.sleep(2)

            print("Iniciando servidor UDP e carga...")
            start_udp_server()
            time.sleep(1)
            start_udp_load(udp_load)
```

# script.py (iv)



```
            print("Iniciando servidor TCP...")
            start_tcp_server()
            time.sleep(1)

            print("Executando carga TCP (30 segundos)...")
            saida_iperf = start_tcp_load()

            f_iperf.write(f"\n=== {alg} BER={ber_texto} Repetição {repeticao}
===\n")
            f_iperf.write(saida_iperf + "\n")

            print("Parando captura tcpdump...")
            stop_tcpdump()
            time.sleep(2)
```

# script.py (v)



```
                print("Parando servidores...")
                stop_servers()

                print("Processando pcap via tcpdump com Regex...")
                bytes_tcp, bytes_totais, eficiencia, tcpdump_output =
process_pcap(pcap_file)

                print("Apagando .pcap para liberar memória da máquina virtual...")
                run(f"sudo himage pc4 rm {pcap_file}")

                vazao_tcp = media_vazao_iperf(saida_iperf)

                print(f">>> Resultados: Bytes Úteis: {bytes_tcp} | Frame Total:
{bytes_totais}")
                print(f">>> Eficiência: {eficiencia:.4f} | Vazão TCP: {vazao_tcp:.2f}
Mbps")
```

# script.py (vi)



```
                # O CSV continua recebendo a variável ber_atual (ex: 1e-05) pro
script de gráficos funcionar!
                writer.writerow([alg, ber_atual, repeticao, bytes_tcp, bytes_totais,
round(eficiencia, 4), vazao_tcp])

print(f"\n🎉 Experimento 100% concluído! Todos os dados estão consolidados no arquivo
'{nome_csv}'.")
```

# analise.py



```python
# ============================================================
# 1. Carregamento dos Dados
# ============================================================
arquivo_alvo = 'resultados_finais_completos.csv'

try:
    df = pd.read_csv(arquivo_alvo)
    print(f"📥 Arquivo '{arquivo_alvo}' carregado com sucesso!")
except FileNotFoundError:
    print(f"❌ Erro: O arquivo '{arquivo_alvo}' não foi encontrado na pasta.")
    exit()

# Ordena os dados pela coluna BER para o eixo X do gráfico ficar na ordem crescente
df = df.sort_values(by='BER')

# ============================================================
```

# analise.py (ii)



```python
# 2. Função para calcular o Intervalo de Confiança (95%)
# ============================================================
def confidence_interval(data, confidence=0.95):
    n = len(data)
    m = np.mean(data)
    std_err = stats.sem(data)
    h = std_err * stats.t.ppf((1 + confidence) / 2, n - 1)
    return h

# ============================================================
# 3. Análise e Geração de Tabelas e Gráficos
# ============================================================
variaveis_de_saida = ['Vazao_TCP_Mbps', 'Eficiencia']

for var in variaveis_de_saida:
    print(f"\n{'='*50}")
```

# analise.py (iii)



```
    print(f"ANÁLISE ESTATÍSTICA: {var}")
    print(f"{'='*50}")

    # Agrupa por Algoritmo e BER e calcula as métricas
    resumo = df.groupby(['Algoritmo', 'BER'])[var].agg(
        Média=np.mean,
        Desvio_Padrão=np.std,
        IC_95=confidence_interval
    ).reset_index()

    # [NOVIDADE]: Cria uma cópia só para a tabela de texto ficar bonita no terminal
    resumo_tabela = resumo.copy()
    resumo_tabela['BER'] = resumo_tabela['BER'].apply(lambda x:
f"10^{int(np.log10(x))}")
    print(resumo_tabela.to_string(index=False))
```

# analise.py (iv)



```
    algoritmos = resumo['Algoritmo'].unique()
    bers = resumo['BER'].unique()

    x = np.arange(len(bers))
    width = 0.35

    fig, ax = plt.subplots(figsize=(8, 6))

    for i, alg in enumerate(algoritmos):
        dados_alg = resumo[resumo['Algoritmo'] == alg]
        medias = dados_alg['Média'].values
        erros = dados_alg['IC_95'].values

        posicao = x - width/2 if i == 0 else x + width/2

        ax.bar(posicao, medias, width, yerr=erros, label=alg.capitalize(),
```

# analise.py (v)



```
                capsize=8, alpha=0.8, edgecolor='black')

    min_val = (resumo['Média'] - resumo['IC_95']).min()
    max_val = (resumo['Média'] + resumo['IC_95']).max()
    margem = (max_val - min_val) * 0.5

    ax.set_ylim(max(0, min_val - margem), max_val + margem)

    # Configuração de Títulos e Eixos
    ax.set_xlabel('Taxa de Erro de Bit (BER)', fontsize=12)
    ax.set_ylabel(f'{var}', fontsize=12)
    ax.set_title(f'Efeito Principal: Algoritmo e BER na {var}\n(Intervalo de
Confiança de 95%)', fontsize=14)
    ax.set_xticks(x)

    # [NOVIDADE]: Renderiza a BER como notação matemática real no eixo X do gráfico
```

# analise.py (vi)

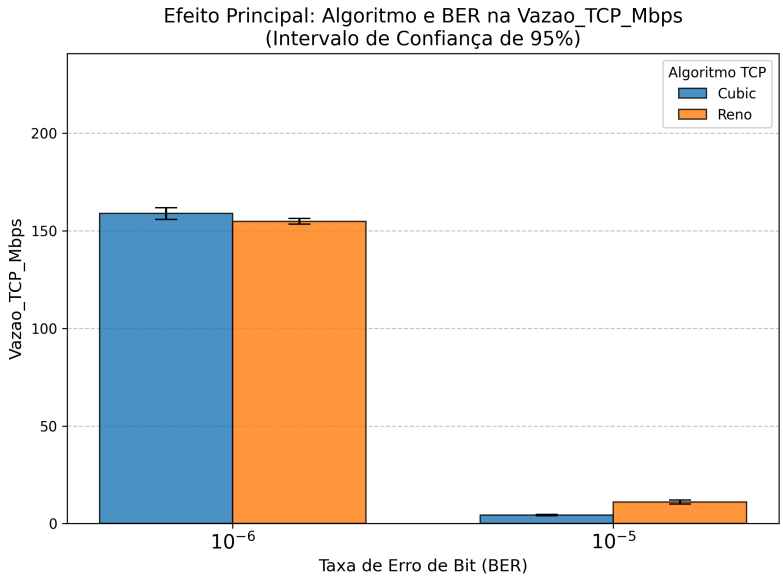


```
(ex: 10^{-5})
    rotulos_ber = [f'$10^{{{int(np.log10(ber))}}}$' for ber in bers]
    ax.set_xticklabels(rotulos_ber, fontsize=14)

    ax.legend(title="Algoritmo TCP", loc='best')

    ax.yaxis.grid(True, linestyle='--', alpha=0.7)
    ax.set_axisbelow(True)

    plt.tight_layout()
    nome_arquivo = f'grafico_{var.lower()}.png'
    plt.savefig(nome_arquivo, dpi=300)
    print(f"[OK] Gráfico gerado e salvo como: {nome_arquivo}")

print("\n🎉 Todos os gráficos e análises foram atualizados com sucesso!")
```

# 4 Resultados

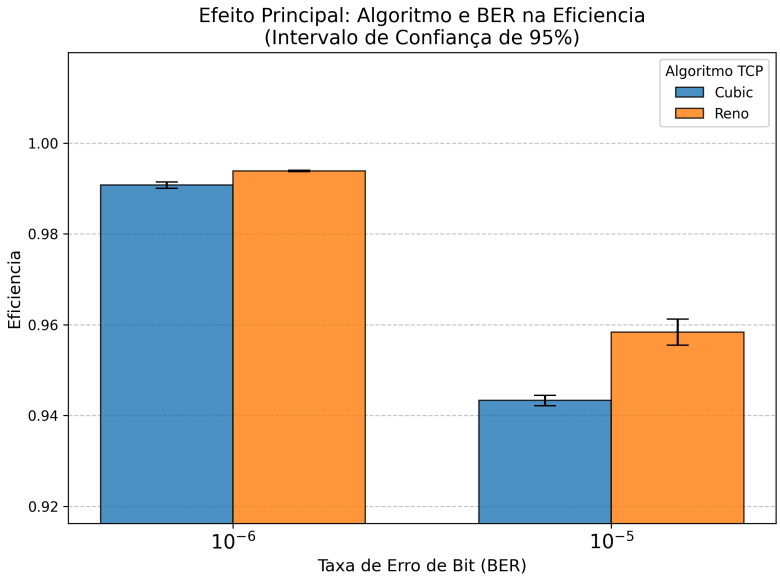
# Vazão Média




Efeito Principal: Algoritmo e BER na Vazao_TCP_Mbps
(Intervalo de Confiança de 95%)
Algoritmo TCP
Cubic
Reno
Vazao_TCP_Mbps
0
50
100
150
200
$10^{-6}$
$10^{-5}$
Taxa de Erro de Bit (BER)

# Eficiência




Efeito Principal: Algoritmo e BER na Eficiencia
(Intervalo de Confiança de 95%)
Algoritmo TCP
Cubic
Reno
1.00
0.98
0.96
0.94
0.92
Eficiencia
$10^{-6}$
$10^{-5}$
Taxa de Erro de Bit (BER)

# Tabela Resumo



| Algoritmo | BER | Vazão Média (Mbps) | Eficiência Média |
|---|---|---|---|
| Cubic | $10^{-6}$ | 158.88 ± 3.05 | 0.9908 ± 0.0007 |
| Cubic | $10^{-5}$ | 4.38 ± 0.33 | 0.9433 ± 0.0012 |
| Reno | $10^{-6}$ | 154.88 ± 1.44 | 0.9939 ± 0.0001 |
| Reno | $10^{-5}$ | 11.09 ± 1.00 | 0.9584 ± 0.0028 |

# 5 Conclusão

# Conclusão



Pelos resultados obtidos podemos observar que quando a rede está sem problemas, os algoritmos não tem uma diferença significativa de desempenho, mas quando temos uma BER pior, podemos ver que o Reno é superior em ambas métricas.